

ROADRESQ

API Specification

Version 1.0 • Production-Oriented API Contract

Backend: FastAPI + Pydantic

Data: PostgreSQL / PostGIS + Redis

Transport: HTTPS JSON + Authenticated WebSockets

AUTH → USER → VEHICLE → REQUEST → MATCH → TRACK → JOB → BILL → PAY → REVIEW

Document Control

Field	Value
Document	API Specification
Product	RoadResQ
Version	1.0
Status	Engineering API baseline
Primary API style	REST/JSON over HTTPS with WebSockets for realtime
Audience	Frontend, mobile, backend, QA, DevOps and integration engineers

1. API Overview

This document defines the public application contract between RoadResQ clients and backend services. It establishes URL conventions, authentication, authorization, request/response formats, resource endpoints, realtime events, errors, pagination, idempotency and integration rules.

1.1 Base URL & Versioning

Production base URL is environment-specific. The API is versioned under a path such as **/api/v1**. Development and staging must use separate hosts/configuration.

Example:

```
https://api.example.com/api/v1/...
```

1.2 Common Headers

Header	Purpose
Authorization	Bearer access token for authenticated endpoints
Content-Type	application/json for JSON bodies
Accept	Expected response media type
Idempotency-Key	Required/recommended for sensitive retryable writes
X-Request-ID	Client-supplied correlation ID; server generates one if absent

1.3 HTTP Methods

Method	Usage
GET	Retrieve resource/collection
POST	Create resource or execute action
PATCH	Partial resource update
PUT	Replace resource when explicitly defined
DELETE	Delete/deactivate resource where permitted

2. Authentication & Authorization

2.1 Authentication Flow

```
Client → Register/Login → OTP/Identity Verification → Access Token + Refresh Token → Authenticated API/WebSocket
```

Endpoint	Purpose	Auth
POST /auth/register	Create account	Public
POST /auth/login	Authenticate account	Public
POST /auth/otp/request	Request OTP	Public / rate-limited
POST /auth/otp/verify	Verify OTP	Public / rate-limited
POST /auth/refresh	Refresh access token	Refresh credential
POST /auth/logout	Invalidate session	Authenticated
GET /auth/me	Current identity	Authenticated

2.2 Roles

Supported roles: **CUSTOMER, PROVIDER, SUPPORT, ADMIN**. Authorization must be enforced server-side for every protected operation.

2.3 Object-Level Authorization

A valid role is not sufficient. The API must verify ownership or an allowed relationship to the requested object. Example: a customer may access only their vehicles and eligible bookings; a provider may access only assigned/authorized jobs.

3. Standard Response Contract

3.1 Success Response

```
{ "data": {...}, "meta": { "request_id": "req_123", "timestamp": "2026-09-04T10:00:00Z" } }
```

3.2 Error Response

```
{ "error": { "code": "BOOKING_INVALID_STATE", "message": "The requested operation is not valid for the current booking state.", "details": {}, "request_id": "req_123" } }
```

3.3 Common HTTP Status Codes

Code	Meaning
200	Successful retrieval/update/action
201	Resource created
202	Accepted for asynchronous processing
204	Successful operation with no response body
400	Invalid request
401	Missing/invalid authentication
403	Authenticated but not authorized
404	Resource not found or intentionally hidden
409	Conflict / state or uniqueness conflict
422	Validation error
429	Rate limit exceeded
500	Unexpected server error
502/503	Upstream/service unavailable

4. Customer APIs

Method	Endpoint	Purpose
GET	/customers/me	Get customer profile
PATCH	/customers/me	Update profile
GET	/customers/me/vehicles	List vehicles
POST	/customers/me/vehicles	Add vehicle
GET	/vehicles/{vehicle_id}	Get vehicle
PATCH	/vehicles/{vehicle_id}	Update vehicle
DELETE	/vehicles/{vehicle_id}	Deactivate vehicle
GET	/customers/me/service-history	View service history

Vehicle creation example:

```
{ "registration_number": "GJXX0000", "make": "Example", "model": "Example Model", "variant": "Base", "fuel_type": "PETROL" }
```

5. Provider APIs

Method	Endpoint	Purpose
POST	/providers	Create provider profile
GET	/providers/me	Get provider profile
PATCH	/providers/me	Update provider profile
POST	/providers/me/documents	Upload verification document
GET	/providers/me/documents	List submitted documents
PATCH	/providers/me/services	Configure services
PATCH	/providers/me/availability	Set availability
PATCH	/providers/me/status	Set online/offline status
GET	/providers/me/earnings	View earnings
GET	/providers/{provider_id}	Public/eligible provider profile

Provider verification lifecycle: **REGISTERED** → **PENDING** → **VERIFIED** → **ACTIVE**; exception states include **REJECTED**, **SUSPENDED**.

6. Booking & Assistance APIs

Method	Endpoint	Purpose
POST	/bookings	Create assistance/service request
GET	/bookings/{booking_id}	Get booking
GET	/bookings	List user's eligible bookings
POST	/bookings/{booking_id}/cancel	Cancel booking
GET	/bookings/{booking_id}/status-history	View state history
POST	/bookings/{booking_id}/retry-match	Retry dispatch where permitted

Create booking request:

```
{ "vehicle_id": "veh_123", "service_type": "TOWING", "location": { "latitude": 22.30, "longitude": 73.18 },  
"description": "Vehicle will not start", "attachments": [] }
```

The backend must validate vehicle ownership, service type, location, duplicate active requests and current account status before creating the booking.

7. Dispatch & Provider Actions

Method	Endpoint	Purpose
GET	/provider/requests	List eligible incoming requests
POST	/provider/requests/{booking_id}/accept	Accept request
POST	/provider/requests/{booking_id}/reject	Reject request
POST	/provider/bookings/{booking_id}/arrive	Mark arrived / verify arrival
POST	/provider/bookings/{booking_id}/start	Start approved work
POST	/provider/bookings/{booking_id}/complete	Complete work

Acceptance must be atomic. If two providers attempt acceptance concurrently, exactly one may win and the other must receive a conflict response.

8. Booking State Machine API Rules

Authoritative lifecycle:

```
REQUESTED → SEARCHING → PROVIDER_ASSIGNED → ACCEPTED → ON_THE_WAY → ARRIVED → INSPECTION → ESTIMATE_PENDING →  
APPROVED → IN_PROGRESS → COMPLETED → INVOICE_GENERATED → PAYMENT_COMPLETED → CLOSED
```

Exception states: CANCELLED, EXPIRED, DISPUTED. The API must reject invalid transitions with a structured 409 response. Clients should never directly set arbitrary status values.

9. Job, Inspection & Estimate APIs

Method	Endpoint	Purpose
POST	/bookings/{booking_id}/inspection	Create inspection
GET	/bookings/{booking_id}/inspection	Retrieve inspection
POST	/bookings/{booking_id}/estimates	Create estimate
GET	/bookings/{booking_id}/estimates/{estimate_id}	Get estimate
POST	/estimates/{estimate_id}/approve	Customer approval
POST	/estimates/{estimate_id}/reject	Customer rejection
POST	/estimates/{estimate_id}/revise	Create revision where permitted

Estimate item example:

```
{ "items": [ {"description": "Part", "quantity": 1, "unit_price": 1500}, {"description": "Labour", "quantity": 1,  
"unit_price": 500} ], "tax_amount": 360, "total_amount": 2360, "currency": "INR" }
```

10. Invoice & Payment APIs

Method	Endpoint	Purpose
POST	/bookings/{booking_id}/invoice	Generate/finalize invoice

Method	Endpoint	Purpose
GET	/invoices/{invoice_id}	Get invoice
POST	/payments	Create payment intent/record
GET	/payments/{payment_id}	Get payment status
POST	/payments/{payment_id}/confirm	Confirm supported payment flow
POST	/payments/webhooks/{provider}	Payment gateway webhook
GET	/providers/me/payouts	Provider payout history

Payment requirements: idempotency keys for payment creation, signed webhook verification, duplicate-event protection, reconciliation for delayed statuses and no client-side declaration of payment success.

MVP payment methods: **UPI + cash recording**. The contract should permit cards and other methods later.

11. Tracking & Location APIs

Method	Endpoint	Purpose
POST	/provider/location	Send provider location update
GET	/bookings/{booking_id}/location	Get current authorized provider location
POST	/bookings/{booking_id}/tracking/start	Start tracking if explicitly required
POST	/bookings/{booking_id}/tracking/stop	Stop tracking

Location payload:

```
{ "latitude": 22.30, "longitude": 73.18, "accuracy_m": 12, "heading": 90, "speed_mps": 8 }
```

Location data must be access-controlled and retained only according to operational/privacy policy.

12. Realtime WebSocket API

WebSocket endpoint example:

```
wss://api.example.com/api/v1/ws/bookings/{booking_id}
```

Event	Direction	Purpose
booking.status.updated	Server → client	Authoritative status change
provider.location.updated	Server → customer	Current provider location
estimate.created	Server → customer	New estimate available
estimate.updated	Server → customer	Estimate revision
payment.updated	Server → client	Payment state change
booking.message.created	Both	Chat message
notification.created	Server → client	Relevant notification

Clients must reconnect safely and fetch the current REST resource after disconnect to reconcile missed events.

13. Notification APIs

Method	Endpoint	Purpose
GET	/notifications	List notifications

Method	Endpoint	Purpose
PATCH	/notifications/{id}/read	Mark notification read
POST	/notifications/preferences	Update notification preferences
GET	/notifications/preferences	Get preferences

Notification delivery itself should be asynchronous and handled by workers. Core booking transactions should not depend on a notification provider being immediately available.

14. Support & Dispute APIs

Method	Endpoint	Purpose
POST	/support/tickets	Create support ticket
GET	/support/tickets	List user's tickets
GET	/support/tickets/{ticket_id}	Get ticket
POST	/support/tickets/{ticket_id}/messages	Add ticket message
POST	/admin/tickets/{ticket_id}/resolve	Resolve ticket
POST	/admin/bookings/{booking_id}/dispute	Flag/open dispute

15. Admin APIs

Method	Endpoint	Purpose
GET	/admin/dashboard	Operational KPIs
GET	/admin/users	Search/manage users
GET	/admin/providers	Search providers
POST	/admin/providers/{id}/verify	Approve verification
POST	/admin/providers/{id}/reject	Reject verification
POST	/admin/providers/{id}/suspend	Suspend provider
GET	/admin/bookings	Monitor bookings
GET	/admin/payments	Payment/reconciliation view
GET	/admin/audit-logs	Audit events

Administrative endpoints require privileged authorization and audit logging. Support agents should receive narrower permissions than full administrators.

16. Pagination, Filtering & Sorting

Collection endpoints should support consistent query parameters:

```
?page=1&page_size=20&sort;=-created_at&status;=COMPLETED
```

Parameter	Rule
page	1-based page number
page_size	Bounded; server enforces maximum
sort	Allow-listed sortable fields only

Parameter	Rule
filter	Allow-listed fields/operators only
search	Validated, bounded text search

Collection response metadata should include total/count where practical, page/page_size and next/previous indicators.

17. Idempotency & Concurrency

Idempotency is mandatory or strongly recommended for operations that may be retried due to network failure: booking creation, provider acceptance, payment creation/confirmation, invoice finalization and selected state-changing actions.

Example:

Idempotency-Key: 7b6c2c3e-...

The server must bind the key to the authenticated actor and operation context and return the original result for safe repeated requests.

Concurrency controls must protect provider assignment, booking transitions, estimate approval and financial operations.

18. Rate Limits & Abuse Protection

Area	Control
Authentication / OTP	Strict per-account/device/IP limits
Booking creation	Prevent excessive active requests and duplicates
Provider location	Bound frequency and payload size
Uploads	Size/type limits and controlled storage
Admin APIs	Strict privileged access and audit
General API	Per-user/IP quotas with configurable thresholds

19. Security Requirements

- All production API traffic uses TLS.
- Access tokens, refresh credentials, OTPs and API secrets are protected and never logged.
- Every protected endpoint enforces authentication and authorization.
- Object-level access checks prevent IDOR-style access to another user's resources.
- Input validation uses Pydantic/domain validation and database constraints.
- File uploads are validated and stored privately.
- Payment webhooks require signature verification and idempotent processing.
- Sensitive admin operations produce audit events.
- CORS, security headers, request-size limits and appropriate CSRF protections are configured.
- Secrets are supplied through secure environment/secret-management mechanisms.

20. API Observability

- Generate a request ID for every API request.
- Log structured method, route, status, latency and safe correlation identifiers.
- Track domain metrics such as booking creation, match success, provider acceptance, cancellations and payment failures.

- Trace external calls to maps, payments, notification providers and storage where feasible.
- Do not log sensitive tokens, OTPs, payment secrets or unnecessary personal data.

21. API Error Catalog

Code	Meaning
AUTH_INVALID_CREDENTIALS	Authentication failed
AUTH_OTP_RATE_LIMITED	OTP request limit reached
FORBIDDEN	Operation not permitted
RESOURCE_NOT_FOUND	Resource unavailable/not visible
VALIDATION_ERROR	Input validation failed
BOOKING_INVALID_STATE	Action conflicts with current state
BOOKING_ALREADY_ASSIGNED	Provider assignment conflict
NO_PROVIDER_AVAILABLE	No eligible provider found
ESTIMATE_APPROVAL_REQUIRED	Customer approval required
PAYMENT_ALREADY_PROCESSED	Payment operation already completed
PAYMENT_RECONCILIATION_PENDING	Gateway result not yet authoritative
IDEMPOTENCY_CONFLICT	Key reused with incompatible request
RATE_LIMITED	Request quota exceeded
INTERNAL_ERROR	Unexpected server failure

22. API Testing Requirements

- Contract tests for every documented endpoint.
- Authentication and RBAC tests for every role.
- Object-level authorization tests using another user's IDs.
- Validation tests for missing, malformed and boundary inputs.
- State-machine transition tests for valid and invalid transitions.
- Race-condition tests for provider acceptance and financial operations.
- Idempotency tests for retried writes/webhooks.
- WebSocket reconnect and missed-event recovery tests.
- Load tests for request/matching/tracking hotspots.
- Security tests for rate limits, upload handling and common API vulnerabilities.

23. OpenAPI Documentation Standard

The FastAPI application should generate an OpenAPI specification from typed Pydantic request/response models. Every endpoint must document authentication, parameters, request schema, success response, error responses and examples. Breaking contract changes require API versioning or a compatible migration strategy.

24. Production API Checklist

- All endpoints have stable IDs/names and documented schemas.

- Authorization is server-side and object-aware.
- Sensitive writes use idempotency where required.
- Booking state transitions are enforced centrally.
- Payment webhooks are verified and replay-safe.
- Realtime events are authenticated and recoverable.
- Rate limits and upload constraints are active.
- Request IDs, logs, metrics and alerts are configured.
- OpenAPI contract is generated and reviewed.
- Integration/E2E/security tests pass before release.

25. API Summary

RoadResQ's API is designed as a versioned, contract-first interface around the core lifecycle: authentication, customer/vehicle management, provider onboarding, dispatch, booking, realtime tracking, job/estimate workflows, billing/payment, notifications, support and administration. PostgreSQL remains authoritative for transactional state, while Redis/WebSockets provide realtime coordination and UX. External services are isolated behind adapters, and all critical writes are designed for authorization, concurrency safety, idempotency and observability.